API Documentation

> ***For LLM agents:*** *this reference is also available as plain text; a compact index is at llms.txt.*

Welcome to the documentation and specification of the LRCLIB API!

This API has generous rate limiting in place and is openly accessible to all users and applications. No API key is required, and anonymous API access remains available.

New update:

- Stricter rate limits now apply to unregistered clients. See Rate Limiting.
- Developers may optionally register applications to receive **higher rate limits** and an API authorization document (useful when submitting to app review).
- A beta `GET /api/v2/search` endpoint is available for lightweight track search without lyrics payloads. See API v2 (beta).

Base URL

All endpoints are served from a single host:

TEXT COPY

```
https://lrclib.net
```

For example, `/api/search` is reachable at `https://lrclib.net/api/search`.

The API is CORS-enabled (`Access-Control-Allow-Origin: *`), so it can be called directly from browser JavaScript without a proxy.

Quickstart

Fetch your first lyrics with a single command:

BASH COPY

```
curl -s 'https://lrclib.net/api/search?q=still+alive' \
  -H 'User-Agent: MyApp v1.0 (https://example.com)'
```

This returns a JSON array of matching records, each shaped like the object described in Lyrics record fields.

Implementation Requirements

Please implement your client responsibly to avoid overloading the server. LRCLIB is a free service, so it relies on clients behaving considerately.

Client identification

We require you to identify your client in requests. Set the `User-Agent` header with your application's name, version, and a link to its homepage or project page, or an email address:

TEXT COPY

```
LRCGET v0.2.0 (https://github.com/tranxuanthang/lrcget)
```

If modifying `User-Agent` is not possible (for example, browser JavaScript restricts this header), you can use `X-User-Agent` or `Lrclib-Client` as alternatives.

Registration is optional: create an application in the developer portal to receive **higher rate limits**, a shareable API authorization document (useful when submitting to app review), and a way for us to contact you if your integration misbehaves. There are two access models:

- **Client-side applications** identify themselves with the public `Lrclib-App-Id` header. The ID is not a secret, so it is safe to ship in a client.
- **Server-side applications** authenticate with a secret `Lrclib-Api-Key` header. Keep the key on your server — never embed it in a client application. Keys can be rotated from the portal by creating a new one and revoking the old one.

Both headers are optional, but invalid values are rejected with `401 Unauthorized` (see Errors). See Rate Limiting for the allowance each model receives.

Rate Limiting

Requests are limited per client. The exact limits are not part of the API contract: the figures below are approximations of the current settings and may change at any time without notice.

How the request identifies itself	Approximate allowance
Generic client (no `User-Agent`, browser, `curl`, or a common HTTP library)	~45 requests/min per IP
Identified client (`User-Agent` naming your application)	~90 requests/min per IP
Registered client-side application (`Lrclib-App-Id`)	~180 requests/min per IP, plus an app-wide allowance (~6,000 requests/min)
Registered server-side application (`Lrclib-Api-Key`)	app-wide allowance only (~6,000 requests/min)

Allowances are per IP address (IPv6 addresses count as `/64` blocks) and refill steadily with a small burst buffer. Anonymous access remains enough for normal use; registered applications receive higher allowances (see Client identification).

When a limit is exceeded, the server responds with `429 Too Many Requests`, a `Retry-After` header specifying how many seconds to wait, and a JSON body naming the limiting scope (`ip` or `application`):

JSON COPY

```
{
  "message": "Too many requests from this IP address. …",
  "name": "RateLimitExceeded",
  "statusCode": 429,
  "scope": "ip"
}
```

Your client **must** honor `Retry-After`. Wait at least the given number of seconds before retrying, and add a small random jitter (for example, up to 30% of the delay) so that many clients throttled at the same moment do not retry in a synchronized wave. Ignoring `Retry-After` and continuing to send requests may result in a temporary ban.

Under extreme floods the network edge may reject requests before they reach the API; those responses are not guaranteed to be JSON. Always rely on the status code and the `Retry-After` header rather than parsing the body.

Request Throttling

To avoid hitting rate limits, send requests sequentially by waiting for each one to finish before starting the next. Additionally, add a short delay between requests (200–500 ms), especially for batch operations like scanning a full music library.

Lyrics record fields

The v1 lyrics endpoints (`/api/get`, `/api/get/{id}` and `/api/search`) use the same JSON object shape:

Field	Type	Description
id	number	Unique, stable ID of the lyrics record
name	string	Alias of `trackName`, kept for backwards compatibility
trackName	string	Title of the track
artistName	string	Name of the artist
albumName	string	Name of the album
duration	number	Track duration in seconds
instrumental	boolean	`true` when the track has no lyrics
hasWordSync	boolean	`true` when `lyricsfile` contains word-level timings (`lines[].words`)
plainLyrics	string | null	Plain, unsynced lyrics. `null` for instrumental records
syncedLyrics	string | null	LRC-formatted lyrics with line timestamps. `null` for instrumental records
lyricsfile	string	Raw Lyricsfile YAML document. Every record is guaranteed to include one

> *The beta `/api/v2/search` endpoint returns a smaller summary object without lyrics text. See v2 search summary fields.*

Errors

Failed requests return a JSON object with the HTTP status code repeated in the `statusCode` field:

JSON COPY

```
{
  "message": "duration: must be between 1 and 3600",
  "name": "ValidationError",
  "statusCode": 400
```

```
}
```

Status	`name`	Meaning
400	`ValidationError`	A parameter failed validation, for example an empty `track_name` or a `duration` outside the allowed range
400	`IncorrectPublishTokenError`	The `X-Publish-Token` header is missing, malformed, already used, or expired
401	`InvalidAppIdError`	The `Lrclib-App-Id` header is not a registered application
401	`MissingApiKeyError`	The `Lrclib-App-Id` belongs to a server-side application, but no `Lrclib-Api-Key` was sent
401	`InvalidApiKeyError`	The `Lrclib-Api-Key` is unknown or revoked, or does not belong to the application in `Lrclib-App-Id`
404	`TrackNotFound`	No lyrics record matched the request
429	`RateLimitExceeded`	Rate limit exceeded; see Rate Limiting. Honor the `Retry-After` header. Rejections at the network edge may not be JSON
500	`UnknownError`	Unexpected server error, try again later
503	`ServerOverloaded`	The server is temporarily busy; retry after the `Retry-After` header (usually 1 second)

Note: Requests with malformed or missing query parameters are rejected by the framework before they reach the API logic and return a plain-text response instead:

TEXT COPY

```
Failed to deserialize query string: missing field `artist_name`
```

Always check the status code and `Content-Type` before assuming the body is JSON.

Supported Endpoints

The endpoints in this section are the stable v1 API. The beta v2 search endpoint is documented separately in API v2 (beta).

Get lyrics with a track's signature

GET `/api/get`

Attempt to find the best match of lyrics for the track. You must provide the track title and the artist name. Providing the album name and the track's duration in seconds is recommended and improves the matching precision.

If no match is found, the API returns `404 Not Found`. Missing tracks can be picked up by the background lyrics fetching service and may become available in a later request.

Note: The provided `duration` is crucial. LRCLIB will attempt to provide the lyrics only when the duration matches the record in LRCLIB's database, or at least with a difference of ±2 seconds in duration.

Query parameters

Field	Required	Type	Description
track_name	true	string	Title of the track
artist_name	true	string	Name of the artist
album_name	false	string	Name of the album (recommended)
duration	false	number	Track's duration in seconds (must be between 1 and 3600)

Example request

HTTP COPY

```
GET /api/get?artist_name=Borislav+Slavov&track_name=I+Want+to+Live&
```

Try this request in your browser

Example response

200 OK:

JSON COPY


```
{
  "id": 3396226,
  "name": "I Want to Live",
  "trackName": "I Want to Live",
  "artistName": "Borislav Slavov",
  "albumName": "Baldur's Gate 3 (Original Game Soundtrack)",
  "duration": 233,
  "instrumental": false,
  "hasWordSync": false,
  "plainLyrics": "I feel your breath upon my neck\n...The clock won
  "syncedLyrics": "[00:17.12] I feel your breath upon my neck\n...[
  "lyricsfile": "version: '1.0'\nmetadata:\n  title: I Want to Live
}
```


See Lyrics record fields for a description of every field. Records published with only `plainLyrics` or `syncedLyrics` are never word-synced, so `hasWordSync` is always `false` for them.

404 Not Found:

JSON COPY

```
{
  "message": "Failed to find specified track",
  "name": "TrackNotFound",
  "statusCode": 404
}
```


Get lyrics by LRCLIB's ID

GET `/api/get/{id}`

Get a lyrics record by the track's absolute ID. Track IDs are returned by other APIs, such as `/api/get` and `/api/search`.

URL parameters

Field	Required	Type	Description
id	true	number	ID of the track

Example request

HTTP COPY

```
GET /api/get/3396226
```

Try this request in your browser

Example response

Please see the `/api/get`'s example response.

Search for lyrics records

GET `/api/search`

Search for lyrics records using keywords. This API returns an array of lyrics records that match the specified search condition(s).

At least ONE of the two parameters, `q` OR `track_name`, must be present. If neither is provided, an empty array is returned.

Note: This API currently returns a maximum of 20 results and does not support pagination. These limitations are subject to change in the future.

Query parameters

Field	Required	Type	Description
q	conditional	string	Search for keyword present in ANY fields (track's title, artist name or album name). When provided, it takes precedence over the structured parameters
track_name	conditional	string	Search for keyword in track's title
artist_name	false	string	Search for keyword in track's artist name
album_name	false	string	Search for keyword in track's album name

Example request

Search for lyrics by using only `q` parameter:

HTTP COPY

```
GET /api/search?q=still+alive+portal
```

Try this request in your browser

Search for lyrics by using multiple fields:

```
HTTP                                                    COPY

GET /api/search?track_name=22&artist_name=taylor+swift
```

Try this request in your browser

Response

JSON array of the lyrics records. Each record follows the shape described in Lyrics record fields.

Publish a new lyrics

POST `/api/publish`

Note: This API is experimental and subject to potential changes in the future.

Publish a new lyrics to LRCLIB database. This API can be called anonymously, and no registration is required.

If `plainLyrics`, `syncedLyrics` and `lyricsfile` are all left empty, the track will be marked as instrumental.

If a `lyricsfile` is provided, it is stored as-is and takes precedence: any `plainLyrics` and `syncedLyrics` in the same request are ignored. A Lyricsfile that contains word-level timings (`lines[].words`) marks the record as word-synced (`hasWordSync: true` in subsequent API responses); records published with only `plainLyrics` or `syncedLyrics` are never word-synced.

All previous revisions of the lyrics will still be kept when publishing lyrics for a track that already has existing lyrics.

Obtaining the Publish Token

Every `POST /api/publish` request must include a fresh, valid Publish Token in the `X-Publish-Token` header. Each Publish Token can only be used once.

The Publish Token consists of two parts: a `prefix` and a `nonce` concatenated with a colon (`{prefix}:{nonce}`).

To obtain a `prefix`, you need to make a request to the `POST /api/request-challenge` API. This will provide you with a fresh prefix string and a target string.

To find a valid `nonce`, you must solve a proof-of-work cryptographic challenge using the provided `prefix` and `target`. For implementation examples, please refer to the source code of LRCGET.

Request header

Header name	Required	Description
X-Publish-Token	true	A Publish Token that can be retrieved via solving a cryptographic challenge

Request JSON body parameters

Field	Required	Type	Description
trackName	true	string	Title of the track
artistName	true	string	Track's artist name
albumName	true	string	Track's album name
duration	true	number	Track's duration

Field	Required	Type	Description
plainLyrics	false	string	Plain lyrics for the track
syncedLyrics	false	string	Synchronized lyrics for the track
lyricsfile	false	string	Raw Lyricsfile YAML for the track. When provided, `plainLyrics` and `syncedLyrics` are ignored

Response

Success response: 201 Created

Failed response (incorrect Publish Token):

JSON COPY

```
{
  "message": "The provided publish token is incorrect",
  "name": "IncorrectPublishTokenError",
  "statusCode": 400
}
```

Flag a lyrics

POST `/api/flag`

Report the currently published lyrics of a track (for example, wrong lyrics, wrong track metadata, or a copyright violation). The flag is recorded against the track's current lyrics.

Every `POST /api/flag` request must include a fresh, valid Publish Token in the `X-Publish-Token` header, obtained the same way as described in the "Publish

a new lyrics" section.

Request header

Header name	Required	Description
X-Publish-Token	true	A Publish Token that can be retrieved via solving a cryptographic challenge

Request JSON body parameters

Field	Required	Type	Description
trackId	true	number	ID of the track to flag
content	false	string	Optional reason for the flag

Example request

HTTP COPY

```
POST /api/flag
X-Publish-Token: VXMwW2qPfW2gkCNSl1i708NJkDghtAyU:0000a1b2c3d4e5f6

{
  "trackId": 3396226,
  "content": "The lyrics don't match the audio"
}
```

Response

Success response: 201 Created

Failed response (incorrect Publish Token):

JSON COPY

```
{
  "message": "The provided publish token is incorrect",
  "name": "IncorrectPublishTokenError",
  "statusCode": 400
}
```


Request a challenge

POST `/api/request-challenge`

Note: This API is experimental and subject to potential changes in the future.

Generate a pair of `prefix` and `target` strings for the cryptographic challenge. Each challenge has an expiration time of 5 minutes.

The challenge's solution is a `nonce`, which can be used to create a Publish Token for submitting lyrics to LRCLIB.

Example response

JSON COPY


```
{
  "prefix": "VXMwW2qPfW2gkCNSl1i708NJkDghtAyU",
  "target": "000000FF0000000000000000000000000000000000000000000000000
}
```

API v2 (beta)

BETA

The v2 API is being introduced with a lightweight search endpoint. It is intended for clients that need to find tracks first and fetch lyrics only for the selected result. The v1 endpoints remain available and unchanged.

> ***Beta notice:*** *the v2 search endpoint and its response shape may change while it is being tested. It is not covered by the v1 compatibility guarantees. Feedback is welcome before the endpoint is finalized.*

v2 search summary fields

The v2 search endpoint returns track metadata and lyrics availability flags, but does not include any lyrics text or Lyricsfile YAML.

Field	Type	Description
id	number	Unique, stable ID of the track
trackName	string	Title of the track
artistName	string	Name of the artist
albumName	string	Name of the album
duration	number	Track duration in seconds
instrumental	boolean	`` `true` `` when the track has no lyrics
hasWordSync	boolean	`` `true` `` when the current lyrics contain word-level timings
hasPlainLyrics	boolean	`` `true` `` when plain lyrics are available
hasSyncedLyrics	boolean	`` `true` `` when LRC-synced lyrics are available
hasLyricsfile	boolean	`` `true` `` when a raw Lyricsfile YAML document is available

To fetch the lyrics for a result, call `GET /api/get/{id}` with the returned `id`. The v1 lyrics record shape is described in Lyrics record fields.

Search for lyrics records (v2 beta)

GET `/api/v2/search`

Search for tracks and return lightweight metadata without lyrics payloads. This endpoint is intended for clients that search first and then request the selected track's lyrics from `/api/get/{id}`.

At least ONE of the two parameters, `q` OR `track_name`, must be present. If neither is provided, an empty array is returned.

The response contains up to 20 results and does not support pagination. The beta response fields are listed in v2 search summary fields; unlike v1 search, there is no `name` alias and no `plainLyrics`, `syncedLyrics`, or `lyricsfile` content.

The same client identification and rate limits described in Implementation Requirements apply.

Query parameters

Field	Required	Type	Description
q	conditional	string	Search for keyword present in ANY fields (track's title, artist name or album name). When provided, it takes precedence over the structured parameters
track_name	conditional	string	Search for keyword in track's title
artist_name	false	string	Search for keyword in track's artist name

Field	Required	Type	Description
album_name	false	string	Search for keyword in track's album name

Example request

HTTP COPY

```
GET /api/v2/search?track_name=I+Want+to+Live&artist_name=Borislav+S
```

Try this request in your browser

Example response

200 OK:

JSON COPY


```
[
  {
    "id": 3396226,
    "trackName": "I Want to Live",
    "artistName": "Borislav Slavov",
    "albumName": "Baldur's Gate 3 (Original Game Soundtrack)",
    "duration": 233,
    "instrumental": false,
    "hasWordSync": false,
    "hasPlainLyrics": true,
    "hasSyncedLyrics": true,
    "hasLyricsfile": true
  }
]
```


To download the selected lyrics, call:

HTTP COPY

```
GET /api/get/3396226
```